

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Chabane
Nom d'usage ▶
Prénom ▶ Alicia
Adresse ▶ La Cerisaie 40 AV Frederic Mistral
06130 Grasse

Titre professionnel visé

Développeur Web et Web Mobile

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

DOSSIER PROFESSIONNEL (DP)

Sommaire

Exemples de pratique professionnelle

Activité-type 1 : Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

p. 5

- ▶ CP 1 Maquetter une application. p. 5
- ▶ CP 2 Réaliser une interface statique et adaptable. p. 12
- ▶ CP 3 Développer une interface utilisateur web dynamique. p. 19
- ▶ CP 4 Réaliser une interface utilisateur avec une solution de gestion de contenu ou e-commerce. p. 24

Intitulé de l'activité-type n° 2 : Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

p.

- ▶ CP 5 Créer une base de données. p. 25
- ▶ CP 6 Développer les composants d'accès aux données. p. 31
- ▶ CP 7 Développer la partie back-end d'une application web ou web mobile. p. 36
- ▶ CP 8 Élaborer et mettre en œuvre des composants dans une application de gestion de contenu ou e-commerce . p. 44

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 45

Déclaration sur l'honneur p. 46

Documents illustrant la pratique professionnelle *(facultatif)* p. 47

Annexes *(Si le RC le prévoit)* p. 48

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 1 ▶ *Maquetter une application.*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Afficher un large catalogue avec filtres : plateforme, genre, nouveautés, promotions

Proposer un panier, la gestion des commandes et un système d'authentification

Mettre en place un espace administrateur pour la gestion des stocks et des produits

Offrir un design responsive, accessible sur mobile, tablette et desktop

Garantir la sécurité des données (RGPD, validation des formulaires, protection contre les attaques courantes)

Intégrer des bonnes pratiques de SEO pour améliorer le référencement

1. Users stories

Suite à cela, pour valider les demandes du client, j'ai rédigé les users stories pour valider les actions que va pouvoir réaliser un visiteur sur le site. Les users stories permettent de comprendre clairement les besoins des utilisateurs de manière concrète, et de s'assurer que les fonctionnalités développées répondent aux attentes des utilisateurs.

En tant que	Je souhaite	Afin de
Utilisateur	m'inscrire sur le site	pour accéder à mon compte.
Utilisateur	me connecter.	à mon espace personnel
Utilisateur	à mon espace personnel	modifier ou supprimer mon profil.
Utilisateur	consulter la liste des produits disponibles.	ajouter des produits à mon panier
Utilisateur	passer commande et payer mes achats.	passer commande et payer mes achats.
Administrateur	accéder à un espace d'administration sécurisé.	pour accéder à mon compte et au tableau bord
Administrateur	gérer la liste des utilisateurs .	(ajouter, modifier, supprimer)
Administrateur	gérer les produits	ajouter, modifier ou supprimer des produits.
Administrateur	gérer la liste des utilisateurs .	(ajouter, modifier, supprimer)

Après un rapide échange de mails avec le client pour vérifier que les users stories étaient bien conformes à ses attentes, j'ai travaillé sur les wireframes du projet.

2.

Wireframes

On peut dire que les wireframes sont des schémas simplifiés de la structuration d'un site web, qui permettent de valider visuellement l'idée que se fait le client de son projet. Les wireframes permettent également d'apporter des précisions sur certains éléments de la structuration d'une page, par exemple les boutons cliquables, les images, etc

Le wireframe se doit d'être simple, généralement en nuance de gris, car il permet simplement de définir la structuration des pages et non pas l'aspect visuel des pages.

Pour ce projet j'ai donc réalisé pour chaque pages du site les wireframes aux formats mobile, tablette et desktop. Pour coller au maximum à l'idée de design mobile-first, j'ai choisi de toujours commencer par le wireframe de la page mobile, puis de continuer sur le format tablette et desktop.

DOSSIER PROFESSIONNEL (DP)

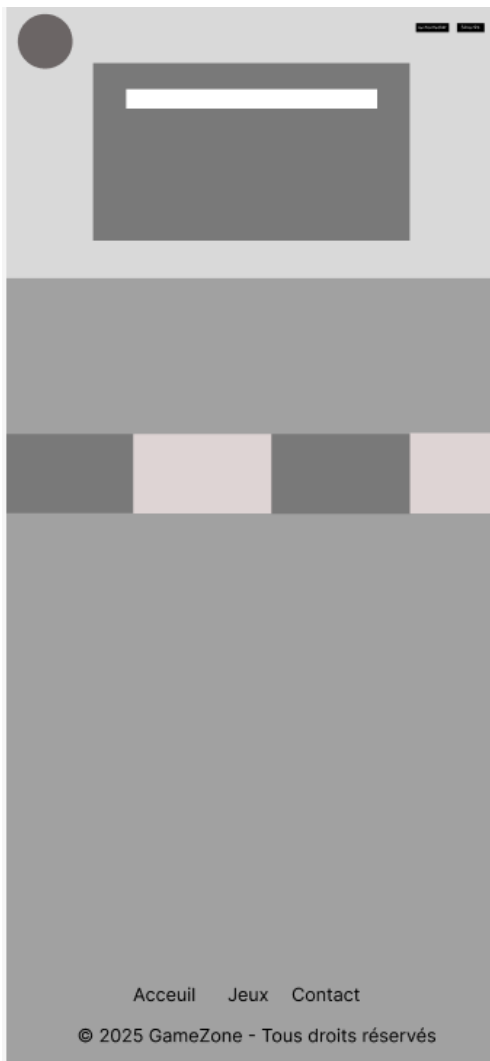
Wireframe général de la page “Home”



On remarque sur cet exemple la représentation des trois formats spécifiés par le client. Je me suis basé sur le format mobile, puis j’ai décliné la structure de la page mobile aux formats tablette et desktop.

On remarque que les éléments ne changent pas, ils sont simplement réorganisés.

Wireframe de la page "Home" - format mobile

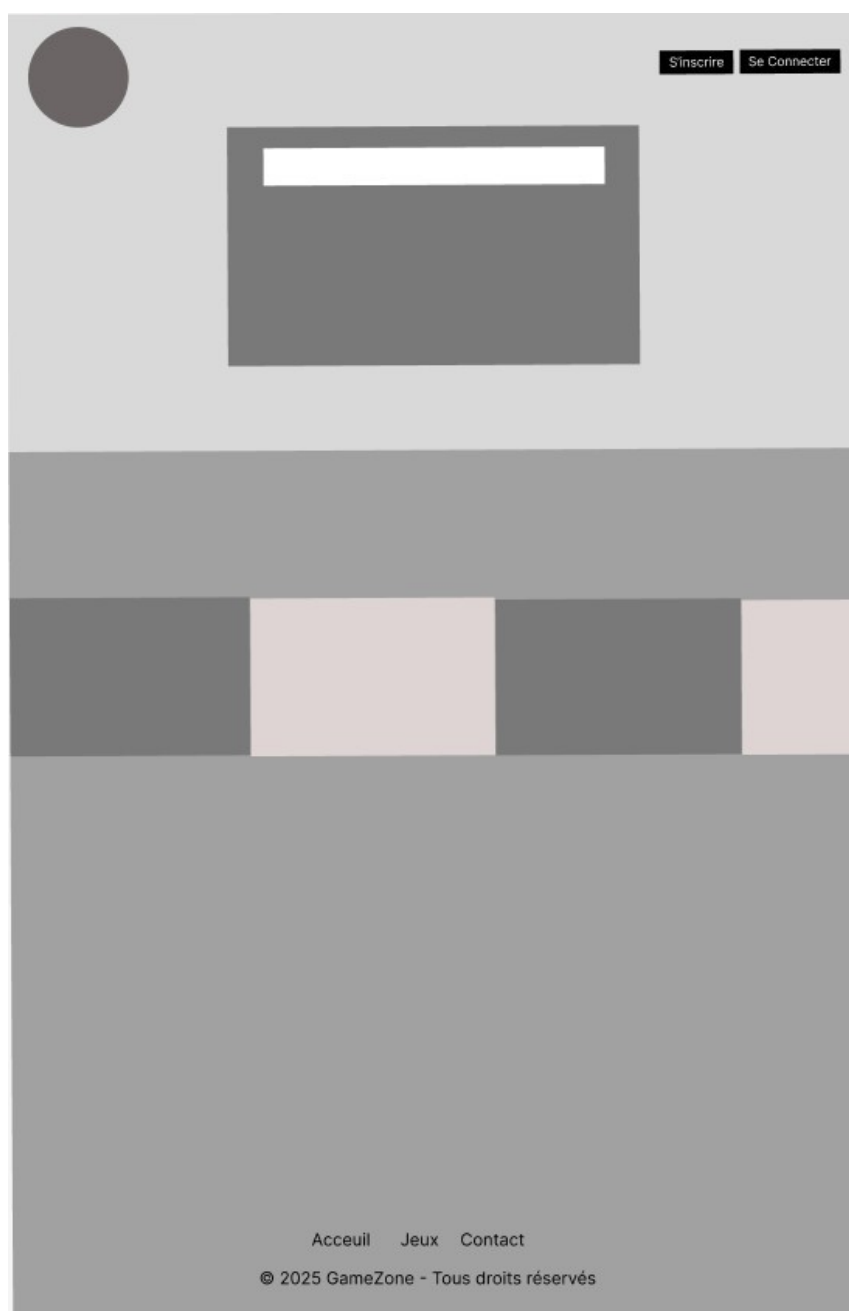


On peut remarquer plusieurs choses sur ce wireframe:

Cette image présente le wireframe de la page d'accueil de l'application en version mobile : on y retrouve l'en-tête avec des boutons de connexion et d'inscription, une zone de recherche, un menu de navigation simplifié en bas de page, ainsi que les informations légales. Les couleurs et formes sont utilisées pour illustrer la répartition des différentes sections

DOSSIER PROFESSIONNEL (DP)

Wireframe de la page "Home" - format tablette



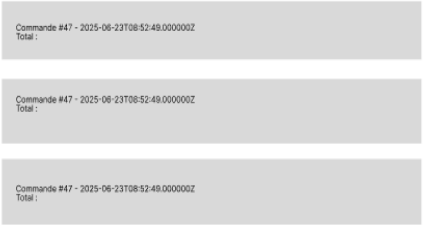
Wireframe de la page “inscription,connexion,et Accueil Connecter” - format desktop



Wireframe de la page “Détail Produit,Panier, Paiement,et Mes commandes” - format desktop



Mes Commandes



Autres exemples de wireframes





DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Dans le cadre de ce projet, j'ai utilisé [Figma](#).

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul pour ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Ecole Laplateforme*

Chantier, atelier, service ► *Projet personnel réalisé en cours de formation.*

Période d'exercice ► Du : *24/06/2024* au : *09/10/2025*

5. Informations complémentaires (facultatif)

Le repo de ce projet est accessible à l'adresse suivante: <https://github.com/aliciachabane/E-commerceJeuxVideo>.

Les documents de conception (users stories et wireframes) sont disponibles dans le dossier "[docs](#)" de ce repo.

L'espace de travail Figma est disponible à cette adresse:

<https://www.figma.com/design/D2REO5C5w280B8ldrf8oaR/>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 2 ► Réaliser une interface web statique et adaptable.

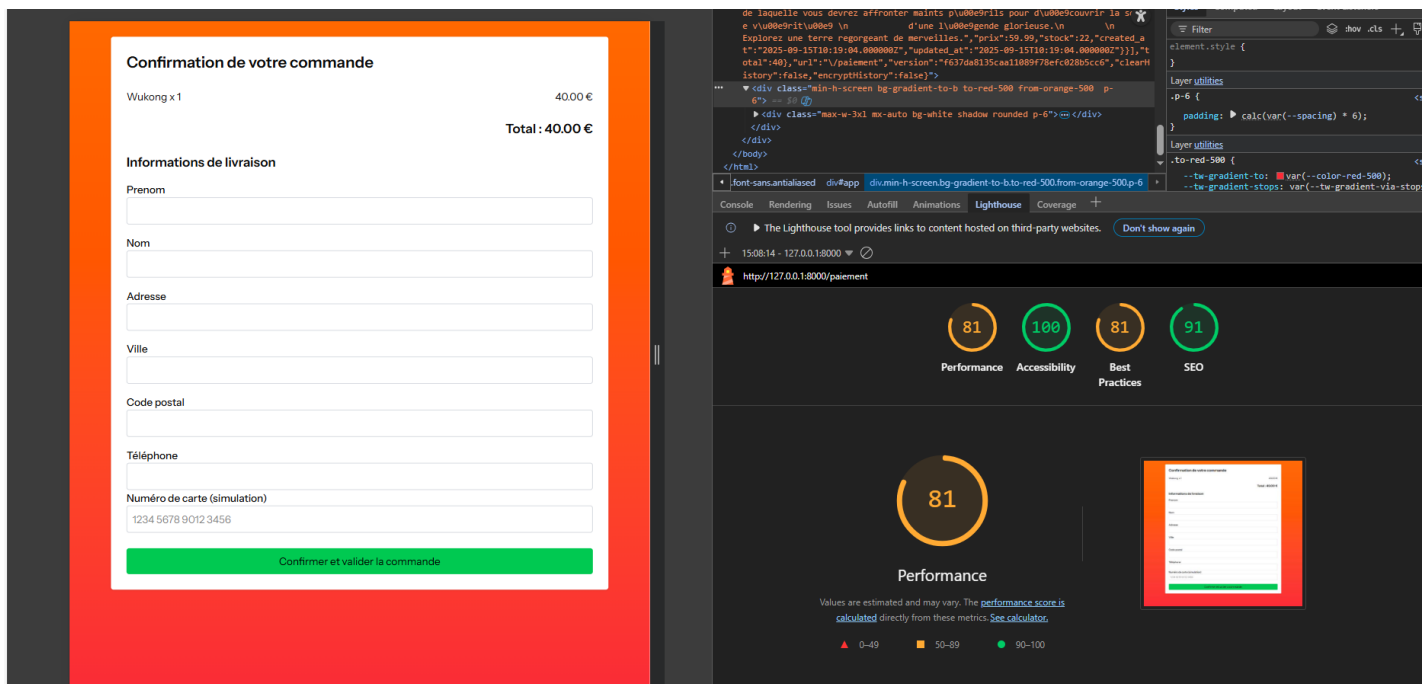
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En reprenant les spécifications du projet (CP1), définies par les demandes du client, l'établissement des users stories et des wireframes, je suis passé à la réalisation du projet.

Ce projet définissait plusieurs objectifs. Le premier d'entre eux était d'être accessible et d'utiliser des bonnes pratiques de SEO. Le second objectif étant de réaliser un site responsive, accessible aux formats mobiles, tablettes et desktop.

La réalisation de ce projet se veut être la plus simple possible. J'ai choisi d'utiliser du HTML, du CSS sans frameworks. J'utilise cependant la librairie [Tailwind.css](#). Concernant l'utilisation de JavaScript, j'ai fait le choix d'en utiliser au minimum. Dans ce projet, JavaScript est uniquement utilisé pour gérer l'ouverture et la fermeture du menu sur mobile.

Pour vérifier que le projet répondait bien à ces impératifs, j'ai choisi d'utiliser l'outil Lighthouse intégré à la console de développement du navigateur.



Rapport Lighthouse - Page de Paiement

Cette analyse montre une performance solide à 81%, traduisant une bonne optimisation du chargement et de l'interactivité. L'accessibilité est optimale à 100%, ce qui garantit une expérience inclusive pour tous les utilisateurs. Les bonnes pratiques sont respectées à 81%, témoignant d'une application correcte des standards modernes de sécurité et de qualité. Enfin, avec 91% en SEO, la page est bien structurée et adaptée pour un bon référencement naturel.



DOSSIER PROFESSIONNEL (DP)

1. Structure des pages

Voici par exemple la structure générale de la page d'accueil:

Pour structurer les pages du projet, pour favoriser l'accessibilité et le SEO, j'ai choisi d'utiliser une bonne sémantique HTML.

Voici par exemple la structure générale de la page d'accueil:

```
<header className="relative">
  
  {!auth?.user && (
    <nav
      className="absolute top-4 right-4 flex flex-col sm:flex-row gap-2 z-20"
      aria-label="Navigation utilisateur">
      <button onClick={() => handleRedirect("/register")} aria-label="S'inscrire"
        className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base">
        S'inscrire
      </button>
      <button onClick={() => handleRedirect("/login")} aria-label="Se connecter"
        className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base">
        Se connecter
      </button>
    </nav>
  )}
</header>
```

la balise <header>, ce qui donne un sens sémantique fort pour les moteurs de recherche.

Le logo a un attribut alt="Logo", ce qui est important pour l'accessibilité et le SEO (même si la valeur pourrait être plus descriptive, ex. « Logo du site [nom] »). J'ai des aria-label sur la navigation et les boutons, ce qui améliore l'expérience pour les lecteurs d'écran. Les images ont l'attribut loading="lazy", ce qui améliore la performance (bon pour le SEO).

```
<main className="relative min-h-screen flex flex-col items-center overflow-x-hidden">
  <ErrorBoundary>
    <Suspense fallback={<div>Chargement du Hero...</div>}>
      <section aria-label="Section héros">
        <Hero />
      </section>
    </Suspense>
  </ErrorBoundary>
```

La structure principale de la page est définie avec la balise <main>, ce qui améliore la sémantique et le référencement naturel. Le Hero est placé dans une section distincte, accompagnée d'attributs ARIA pour l'accessibilité. L'utilisation de Suspense et ErrorBoundary permet d'assurer une meilleure expérience utilisateur en gérant le chargement et les éventuelles erreurs. Pour renforcer le SEO, cette section devrait intégrer un titre visible (<h1>) et du contenu textuel pertinent, afin que les moteurs de recherche identifient plus clairement le sujet de la page. »

```

101 {results.length === 1 && (
102   <article className="max-w-xl mx-auto bg-white text-black rounded-lg shadow-lg p-6 mt-8">
103     <h2 className="text-2xl font-bold mb-4">{results[0].nom}</h2>
104     <img
105       src={`/${products}/${results[0].image}`}
106       alt={results[0].nom}
107       className="w-full h-auto object-cover rounded mb-4"
108       loading="lazy"/>
109     <p className="mb-2 text-gray-700">{results[0].description}</p>
110     <p className="text-red-500 font-bold text-xl mb-2">{results[0].prix} €</p>
111     <button
112       onClick={() => { setResults([]); setHasSearched(false); }}
113       className="mt-6 bg-orange-400 text-white px-4 py-2 rounded shadow">
114       Retour à l'accueil
115     </button>
116   </article>
117 )}

```

La fiche produit est encapsulée dans une balise <article>, ce qui lui confère une autonomie sémantique utile pour le référencement. Le titre du produit est mis en évidence dans une balise de titre, et l'image bénéficie d'un attribut alt descriptif pour l'accessibilité et le SEO. Le prix est clairement affiché et mis en valeur visuellement. Un bouton permet de revenir à l'accueil, mais il serait préférable de le transformer en lien <a> pour respecter les bonnes pratiques de navigation. Afin d'optimiser davantage le SEO, on pourrait utiliser le balisage structuré Schema.org (Product et Offer) et enrichir le contenu textuel avec des mots-clés pertinents. »

```

118 {hasSearched && results.length === 0 && (
119   <section aria-live="polite" className="text-center text-red-500 mt-8">
120     Produit indisponible
121     <button
122       aria-label="Retour à l'accueil"
123       onClick={() => { setResults([]); setHasSearched(false); }}
124       className="mt-4 bg-orange-400 text-white px-4 py-2 rounded shadow">
125       Retour à l'accueil
126     </button>
127   </section>
128 )}
129
130 {(!hasSearched || results.length !== 1) && (
131   <h1 className="text-3xl font-bold mb-4 mt-8 z-10">Bienvenue</h1>
132
133   <ErrorBoundary>
134     <Suspense fallback={<div>Chargement du slider...</div>}>
135       <section aria-label="Slider de produits" className="w-full mt-8 gap-4 z-10">
136         <Slider {...settings}>
137           
138           
139           
140           
141         </Slider>
142       </section>
143     </Suspense>
144   </ErrorBoundary>
145
146   <section className="mt-6 z-10" aria-label="Message utilisateur">
147     {auth?.user ? (
148       <p className="text-xl">Bonjour, {auth.user.name}</p>
149     ) : (
150       <p className="text-xl">Vous n'êtes pas connecté.</p>
151     )}
152   </section>
153 )}
154 </main>

```

balise <section> avec aria-live="polite". Cela signifie que les lecteurs d'écran annoncent automatiquement le changement → très bon pour l'accessibilité. Le bouton de retour à l'accueil a un aria-label, ce qui clarifie son rôle. Le titre principal de la page est un <h1> (Bienvenue), ce qui donne une bonne base sémantique pour le SEO. Le slider de produits est isolé dans une <section> avec aria-label="Slider de produits", ce qui aide à comprendre sa fonction. Les images du slider ont des attributs alt et sont en lazy-loading → bon pour l'accessibilité et la performance. Le message utilisateur (connecté ou non) est mis dans une <section> dédiée, ce qui garde la structure claire.

```

150
151 <footer>
152   <NavFooter items={footerItems} />
153 </footer>
154 </>
155 );
156 }
157 Accueil.layout = page => <AppLayout>{page}</AppLayout>;
158

```

j'utilise la balise <footer>, ce qui est sémantiquement correct → ça indique la zone de bas de page, bien reconnue par les moteurs de recherche et les technologies d'assistance. J'inclus un composant NavFooter, pour une navigation de liens (mentions légales, contact, réseaux sociaux, etc.) → très utile pour l'accessibilité et le SEO interne (maillage). La présence d'une navigation dédiée dans le footer aide Google à comprendre la structure du site et à indexer les pages importantes.

2. responsive et media queries

La partie dynamique et responsive de mon site est gérée directement par Tailwind CSS et les breakpoints intégrés (sm:, md:, lg:...), ce qui me permet d'adapter l'affichage aux différentes tailles d'écran sans écrire de @media manuels. Le slider utilise aussi sa propre configuration responsive pour ajuster automatiquement le nombre d'éléments affichés.

Accueil Non connecter

```
{auth.user || {} }
<nav
  className="absolute top-4 right-4 flex flex-col sm:flex-row gap-2 z-20"
  aria-label="Navigation utilisateur">
  <button onClick={() => handleRedirect("/register")} aria-label="S'inscrire"
    className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base">
    S'inscrire
  </button>
  <button onClick={() => handleRedirect("/login")} aria-label="Se connecter"
    className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base">
    Se connecter
  </button>
</nav>
```

En mobile (flex-col) les boutons s'empilent verticalement.

En tablette/desktop (sm:flex-row) ils passent côte à côte.

La taille du texte et des paddings s'adapte aussi (text-sm → sm:text-base, px-3 py-1 → sm:px-4 sm:py-2).

```
44 const settings = {
45   dots: true,
46   infinite: true,
47   speed: 500,
48   slidesToShow: 3,
49   slidesToScroll: 1,
50   autoplay: true,
51   autoplaySpeed: 2000,
52   arrows: false,
53   responsive: [
54     { breakpoint: 1024, settings: { slidesToShow: 2 } },
55     { breakpoint: 640, settings: { slidesToShow: 1 } }
56   ],
```

le nombre d'images affichées varie automatiquement :

3 sur desktop,

2 sur tablette,

1 sur mobile.

Page Accueil Connecter

L'adaptabilité de mon interface est assurée directement par les breakpoints Tailwind. Par exemple, la grille de produits utilise `grid-cols-1 sm:grid-cols-2 md:grid-cols-4`, ce qui permet d'afficher automatiquement 1, 2 ou 4 produits par ligne selon la taille de l'écran, sans écrire de `@media`. De la même manière, les composants de navigation et la typographie s'ajustent avec des classes comme `sm:` ou `md:`, garantissant une expérience utilisateur fluide sur mobile, tablette et desktop.

```
<h1 className="text-3xl font-bold mb-4">Bienvenue sur votre espace</h1>
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 gap-6">
  {filteredProducts.length > 0 ? (
    filteredProducts.map((produit) => (
      <div
        key={produit.id}
        className="bg-white text-black p-4 rounded-lg shadow-lg hover:shadow-xl transition cursor-pointer"
        onClick={() => handleRedirect(route('produits.show', produit.id))}>
        <h3 className="text-xl font-semibold mb-2">{produit.nom}</h3>
        <p className="text-red-500 font-bold mb-2">Prix : {produit.prix} €</p>
        <p className="text-sm text-gray-500">Catégorie : {produit.category?.nom ?? 'Non classé'}</p>
        <div className="mt-2">
          <p className="text-sm font-medium">Plateformes disponibles :</p>
          {produit.plateformes.map((plateforme) => (
            <p key={plateforme.id} className="text-gray-600">
              {plateforme.nom} - Prix : {plateforme.pivot?.prix ?? produit.prix} €
            </p>
          ))}
        </div>
        {produit.image && (
          <div className="mt-4">
            <img
              src={`~/produits/${produit.image}`}
              alt={produit.nom}
              className="w-full h-auto object-cover rounded"/>
            </div>
          </div>
        </div>
      </div>
    )
  ) : null}
```

Page DetailProduit

Container principal centré et adaptatif

```
<div className="relative z-10 w-full max-w-lg mx-auto flex flex-col items-center mt-[10em] pb-10">
```

`w-full` → prend toute la largeur sur mobile.
`max-w-lg mx-auto` → limite la largeur sur desktop et centre le contenu.
`flex flex-col items-center` → aligne verticalement les éléments, ce qui reste naturel sur toutes les tailles d'écran.

```
<img
  src={`~/produits/${produit.image}`}
  alt={produit.nom}
  className="mt-4 w-full max-w-md rounded"/>
```

`w-full` → image responsive qui remplit la largeur du container.
`max-w-md` → limite la largeur sur écran large pour éviter qu'elle devienne trop grande.

```
<input
  type="number"
  min="1"
  value={quantite}
  max={produit.stock}
  onChange={handleQuantiteChange}
  className="mt-4 px-2 py-1 rounded text-center w-20 bg-white text-black"/>

<button
  onClick={handleAjouterAuPanier}
  className="bg-amber-400 mt-4 text-black px-4 py-2 rounded">
  Ajouter au panier
</button>
```

Les champs et boutons sont centrés, ont des padding adaptatifs et restent lisibles sur mobile et desktop.

DOSSIER PROFESSIONNEL (DP)

```
export function NavMain({ items = [], onPlatformClick, onSearch, results = [] }) {  
  const page = usePage();  
  const [showInput, setShowInput] = useState(false);  
  const [searchValue, setSearchValue] = useState("");  
  const inputRef = useRef(null);  
  
  useEffect(() => {  
    if (showInput && inputRef.current) {  
      inputRef.current.focus();  
    }  
  }, [showInput]);  
}
```

```
  onClick={() => setShowInput(true)}
```

PoDans ce composant, une variable d'état showInput (initialisée à false) contrôle l'affichage de la zone de recherche.

Lorsqu'on clique sur l'icône de recherche, setShowInput(true) est appelé et le champ s'affiche.

Une fois la recherche validée, setShowInput(false) est rappelé, ce qui masque à nouveau l'input.

Ainsi, l'affichage du champ est entièrement dynamique et géré par l'état React.ur réaliser ce projet, j'ai utilisé **VSCode** avec **Prettier** pour formater le code

2. Précisez les moyens utilisés :

Pour vérifier l'accessibilité et les bonnes pratiques de SEO, j'ai utilisé l'outil **Lighthouse** de la console

Pour rendre le site dynamique et responsive, j'ai utilisé la gestion d'état avec React (useState)

pour contrôler l'affichage conditionnel (par exemple, la zone de recherche), des gestionnaires

d'événements (onClick, onSubmit) pour déclencher ces changements,

ainsi que les classes responsives de Tailwind CSS (telles que sm:, md:)

afin d'adapter la mise en page selon la taille d'écran, sans avoir recours à des @media manuels.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *École Laplateforme*

Chantier, atelier, service ► *Projet personnel réalisé en cours de formation.*

Période d'exercice ► Du : *24/06/2024* au : *09/10/2025*

5. Informations complémentaires (facultatif)

Le repo du projet est accessible ici: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Le projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 3 ► Développer une interface utilisateur web dynamique.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ce titre professionnel, j'ai réalisé une application de gestion de budget simplifiée. Cette application permet de gérer les dépenses et les revenus d'un utilisateur connecté. L'utilisateur peut créer différents budgets, ainsi que différentes catégories de dépenses. Il peut également ajouter des dépenses et des revenus à ces budgets.

Pour réaliser la partie front-end de l'application, j'ai utilisé la librairie React en y ajoutant les modules suivants:

- **Gestion des routes:** [gérées par Laravel + Inertia](#)
- **Gestion du state:** [React natif \(useState, useReducer\)](#)
- **Gestion des requêtes:** [@inertiajs/inertia](#), [@inertiajs/react](#) (Inertia gère navigation et HTTP)
- **Gestion des formulaires:** [useForm](#) de [@inertiajs/react](#)
- **Gestion des headers HTTP:** [Automatique via Inertia \(X-Inertia, X-CSRF-TOKEN\)](#)
- **Librairie de composants:** [@headlessui/react](#), [@radix-ui/react-*](#), [lucide-react](#), [react-icons](#), [react-slick](#), [qrcode.react](#), + [TailwindCSS](#)

Accueil non connecter redirection vers login ou register

```
{
  render: user ? user : null,
  <nav>
    className="absolute top-4 right-4 flex flex-col sm:flex-row gap-2 z-20"
    aria-label="Navigation utilisateur"
    <button onClick={() => handleRedirect("/register")} aria-label="S'inscrire"
      className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base" >
      S'inscrire
    </button>
    <button onClick={() => handleRedirect("/login")} aria-label="Se connecter"
      className="bg-orange-300 text-black px-3 py-1 sm:px-4 sm:py-2 rounded-md shadow text-sm sm:text-base" >
      Se connecter
    </button>
  </nav>
```

Accueil Connecter redirection vers Detail Produit

```
<div>
  key={produit.id}
  className="bg-white text-black p-4 rounded-lg shadow-lg hover:shadow-xl transition cursor-pointer"
  onClick={() => handleRedirect(route('produits.show', produit.id))}>
```

Detail Produit redirection vers Panier

```
const handleAjouterAuPanier = () => {
  Inertia.post(route('panier.ajouter'), {
    produit_id: produit.id,
    prix: prixParPlateforme[plateformeSelectionnee] ?? produit.prix,
    quantite: quantite,
  }, {
    onSuccess: () => Inertia.visit(route('panier.afficher'))
  });
};

<button
  onClick={handleAjouterAuPanier}
  className="bg-amber-400 mt-4 text-black px-4 py-2 rounded" >
  Ajouter au panier
</button>
```

Panier redirection vers Paiement

```
<button
  onClick={() => Inertia.visit(route('paiement.page'))}
  className="bg-amber-400 px-4 py-2 rounded text-white font-semibold hover:bg-amber-400" >
  Paiement
</button>
```

Paiement redirection vers Mes commandes

```
const handleConfirmerCommande = () => {
  Inertia.post(route('paiement.valider'), form, {
    onSuccess: () => {
      Inertia.visit(route('commandes.mes'));
    }
  });
};

function handleDelete(id) {
  router.delete(`/panier/${id}`, {
    preserveScroll: true,
  });
}
```

Le routeur de mon projet est géré par Inertia.js, qui fait le lien entre les routes Laravel et les composants React. Cela permet de naviguer entre les pages (Accueil, Connexion, Produits, Panier, etc.) sans rechargement complet, en offrant une expérience fluide similaire à une SPA. J'utilise par exemple Inertia.visit pour rediriger l'utilisateur et router.post pour gérer des actions comme l'ajout au panier.

```
Route::get('/', function () {
  return Inertia::render('Accueil');
})->name('home');

Route::middleware(['auth', 'verified'])->group(function () {
  Route::get('/accueil-connecter', function () {
    $produits = Produits::with(['category', 'plateformes'])->get();
    return Inertia::render('AccueilConnecter', [
      'produits' => $produits,
      'auth' => ['user' => auth()->user()],
    ]);
  })->name('accueil.connecter');
});

Route::middleware('auth')->post('/ajouter-au-panier', [PanierController::class, 'ajouterAuPanier'])->name('panier.ajouter');

Route::get('/panier', [PanierController::class, 'afficher'])->name('panier.afficher');

Route::delete('/panier/{id}', [PanierController::class, 'supprimer'])->name('panier.supprimer');

Route::middleware(['auth'])->group(function () {
  Route::get('/paiement', [CommandeController::class, 'afficherPagePaiement'])->name('paiement.page');
  Route::post('/paiement/valider', [CommandeController::class, 'validerCommande'])->name('paiement.valider');
});

Route::get('/mes-commandes', [CommandeController::class, 'mesCommandes'])->name('commandes.mes');
```

2. Les requêtes

Pour effectuer les requêtes, j'ai commencé par définir une fonction (méthode) dans mon Controller

```
public function register(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|confirmed|min:6',
        'role' => 'nullable|in:user,admin',
        'admin_code' => 'nullable|string|max:255|required_if:role,admin',
    ]);

    $role = 'user';
    if ($request->role === 'admin' && $request->admin_code === env('ADMIN_REGISTRATION_CODE')) {
        $role = 'admin';
    }

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'role' => $role,
    ]);

    Auth::login($user);

    return $role === 'admin'
        ? redirect()->route('dashboard')
        : redirect()->route('accueil.connecter');
}
```

Dans mon projet, j'ai implémenté plusieurs requêtes, notamment la création d'un utilisateur (INSERT) lors de l'inscription et la vérification des identifiants (SELECT) lors de la connexion, en utilisant Eloquent de Laravel et l'authentification intégrée.

```

43
44     public function showLoginForm()
45     {
46         return Inertia::render('auth/Login');
47     }
48
49     public function showRegisterForm()
50     {
51         return Inertia::render('auth/Register');
52     }
53
54     public function loginSubmit(Request $request)
55     {
56         $request->validate([
57             'email' => 'required|email',
58             'password' => 'required|min:6',
59             'role' => 'required|in:user,admin',
60         ]);
61
62         $user = User::where('email', $request->email)->first();
63
64         if (!$user || !Hash::check($request->password, $user->password)) {
65             return Inertia::render('auth/Login', [
66                 'error' => 'Email ou mot de passe incorrect.',
67             ]);
68         }
69
70         if ($user->role !== $request->role) {
71             return Inertia::render('auth/Login', [
72                 'error' => 'Le rôle sélectionné ne correspond pas à votre compte.',
73             ]);
74         }
75
76         Auth::login($user);
77
78         return $user->role === 'admin'
79             ? redirect()->route('dashboard')
80             : redirect()->route('accueil.connecter');
81     }
82
83     public function logout(Request $request)
84     {
85         Auth::logout();
86         $request->session()->invalidate();
87         $request->session()->regenerateToken();
88     }

```

DOSSIER PROFESSIONNEL (DP)

Laravel (qui utilise l'ORM Eloquent) pas besoin de faire une requête avec SELECT*

2. Précisez les moyens utilisés :

Pour réaliser ce projet j'ai utilisé [Vite](#), ainsi que les documentations des différentes dépendances de mon projet, notamment les documentations de React Redux, et de React Query

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *École Laplateforme*

Chantier, atelier, service ► *Projet personnel réalisé en cours de formation.*

Période d'exercice ► Du : *24/06/2024* au : *09/10/2025*

5. Informations complémentaires (facultatif)

Le repo de ce projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Le projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 4 ► Réaliser une interface utilisateur avec une solution de gestion de contenu ou e-commerce.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Import de composant, créer mes pages et afficher les données ,responsif et dynamique

2. Précisez les moyens utilisés :

React,tailwind,composant ui

3. Avec qui avez-vous travaillé ?

J'ai travailler seul sur ce projet

4. Contexte

Nom de l'entreprise, organisme ou association ►

Chantier, atelier, service►

Période d'exercice ► Du : 24/06/2024 au : 09/10/2025

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d’une application web ou web mobile en intégrant les recommandations de sécurité.

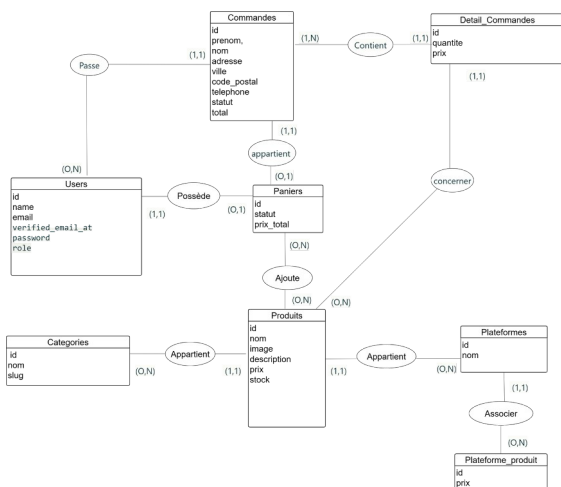
CP 5 ▶ Créer une base de données.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour mieux comprendre la structure de mon projet, j’ai donc créé un MCD (*Modèle Conceptuel des Données*).

1. Le MCD

Schéma conceptuel de la BDD



Pas de relation entre users et produit il s'exprime via la relation entre Utilisateur et Panier, puis entre Panier et Produit

Après avoir défini la représentation visuelle de ma base de données avec le MCD, j'ai créé un **MLD** (*modèle logique des données*) pour représenter les relations entre les tables.

MLD

Users : id(PK),name,email,password,role, created_at,updated_at

Produits : id (PK),nom,image,description,prix,stock,categorie_id (FK → Categories.id)deleted_at

Categories : id (PK), nom,slug,parent_id (FK →Categories.id, optionnel)

Plateformes : id (PK),nom

Plateformes_Produits : id (PK),produit_id (FK → Produits.id),plateforme_id (FK → Plateformes.id),prix

Paniers : id (PK), user_id (FK → Users.id), statut, prix_total

Commandes : id (PK),user_id (FK → Users.id),prenom,nom,adresse,ville,code_postal,telephone,statut,total

Detail_Commandes : id (PK),commande_id (FK → Commandes.id),produit_id (FK →Produits.id),quantite,prix

Produits : belongsTo(Categories::class, 'category_id')

belongsToMany(Plateforme::class, 'plateforme_produit', 'produit_id', 'plateforme_id')->withPivot('prix')

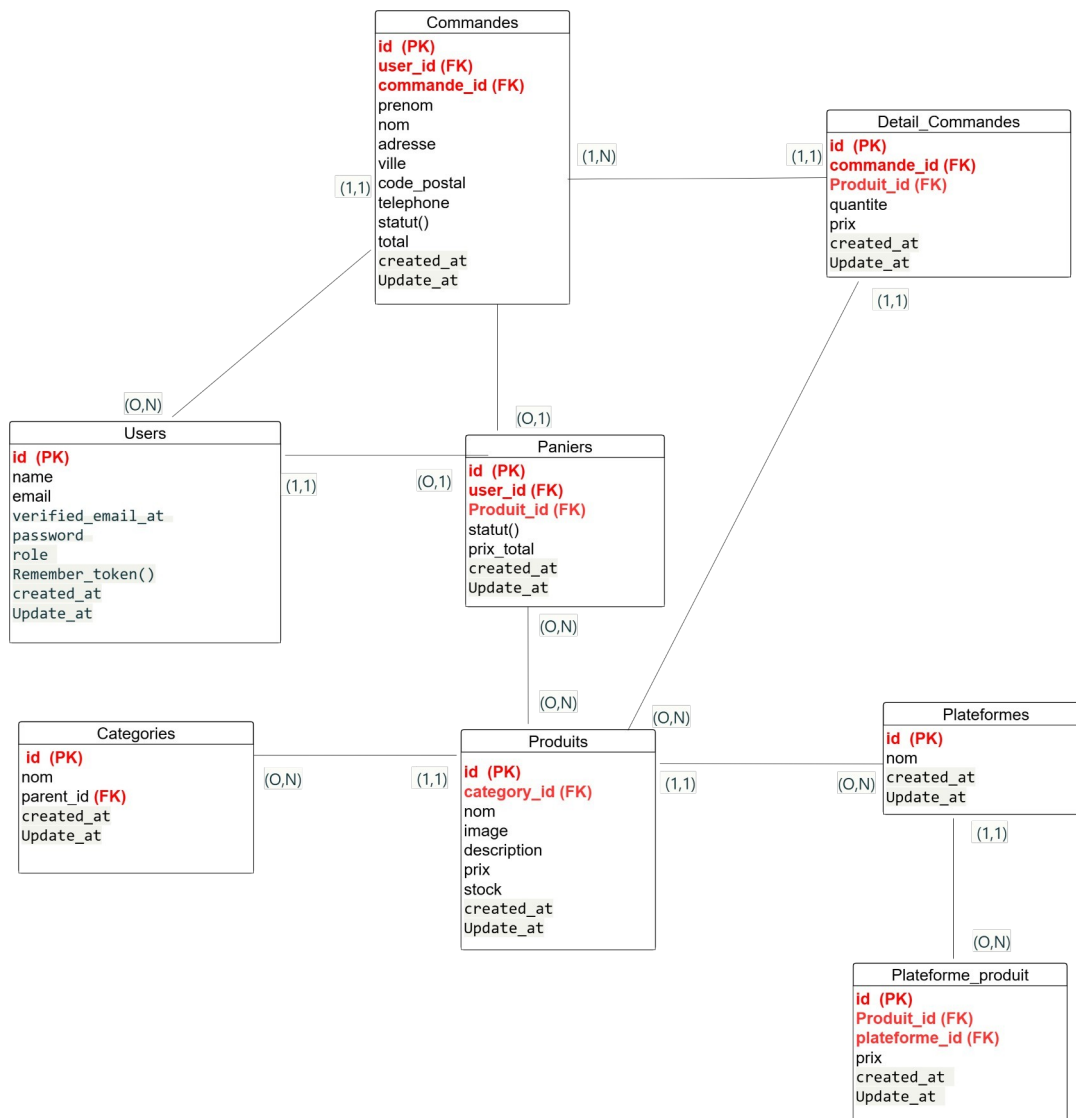
Plateformes : belongsToMany(Produits::class, 'plateforme_produit', 'plateforme_id', 'produit_id') ->withPivot('prix')

Paniers : belongsTo(User::class);,belongsTo(Produits::class);,belongsTo(Commandes::class, 'commande_id');

detail__commandes->belongsTo(Commandes::class, 'commande_id');->belongsTo(Produits::class);

commandes->belongsTo(Commandes::class, 'commande_id');->hasMany(DetailCommandes::class, 'commande_id');
->hasMany(Paniers::class);

Categories : ->hasMany(Categories::class, 'parent_id');->hasMany(Produits::class);->belongsTo(Categories::class, 'parent_id');

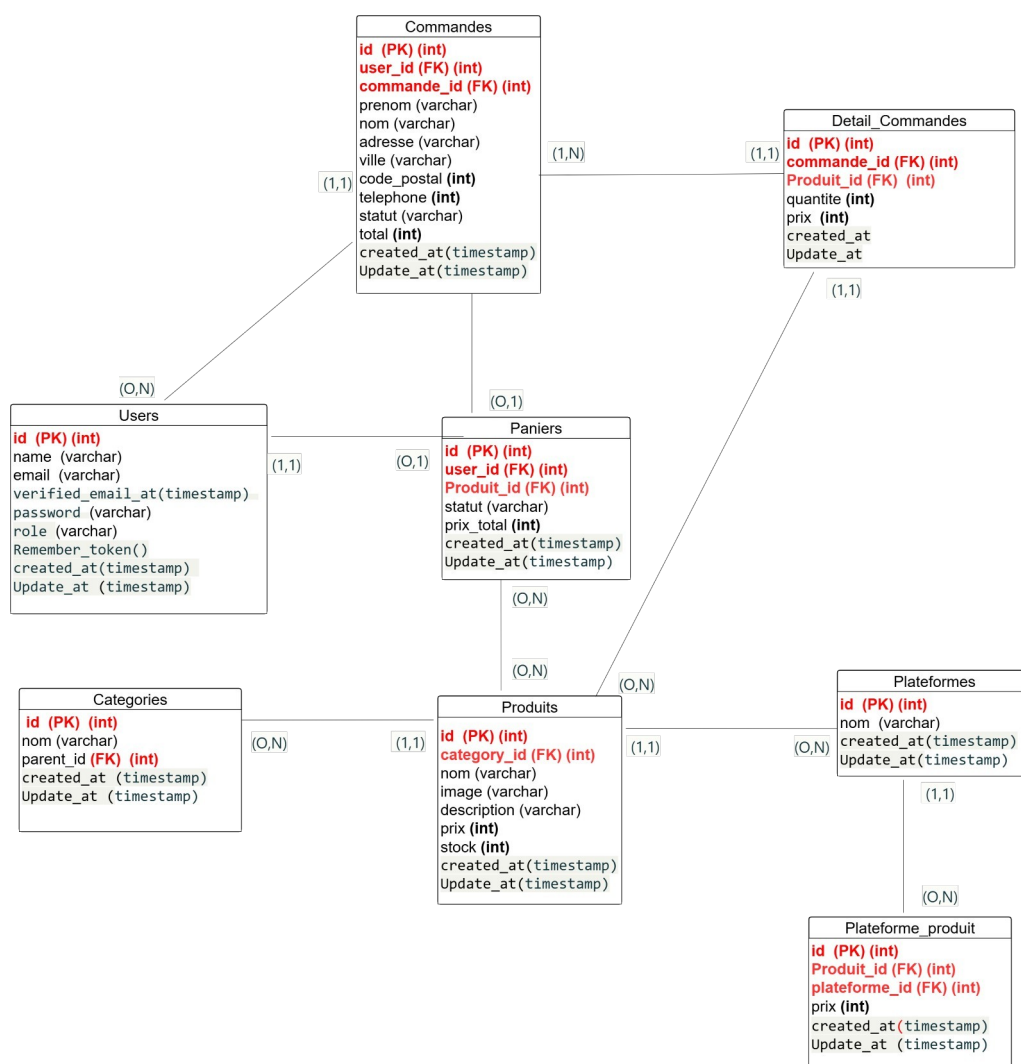


DOSSIER PROFESSIONNEL (DP)

Puis une fois le dictionnaire des données réalisé, je me suis servi de **PgAdmin** pour réaliser le MPD (*modèle physique des données*).

4. Le MPD

représentent l'intégralité du MPD.



```

2 CREATE TABLE User (
3     id_user INT AUTO_INCREMENT PRIMARY KEY,
4     rôle ENUM('client', 'administrateur'),
5     nom VARCHAR(100),
6     prénom VARCHAR(100),
7     email VARCHAR(255) UNIQUE,
8     mot_de_passe VARCHAR(255),
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
11 );
12
13 -- Table Catégories
14 CREATE TABLE Catégories (
15     category_id INT AUTO_INCREMENT PRIMARY KEY,
16     nom VARCHAR(100),
17     description TEXT,
18     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
19     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
20 );
21
22 -- Table Produits
23 CREATE TABLE Produits (
24     id_produit INT AUTO_INCREMENT PRIMARY KEY,
25     nom VARCHAR(100),
26     image VARCHAR(255),
27     description TEXT,
28     prix DECIMAL(10, 2),
29     stock INT,
30     category_id INT,
31     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
33     FOREIGN KEY (category_id) REFERENCES Catégories(category_id)
34 );
35
36 -- Table Panier
37 CREATE TABLE Panier (
38     id INT AUTO_INCREMENT PRIMARY KEY,
39     id_user INT,
40     id_produit INT,
41     quantité INT,
42     statut VARCHAR(20),
43     date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
44     prix DECIMAL(10, 2),
45     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
46     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
47     FOREIGN KEY (id_user) REFERENCES User(id_user),
48     FOREIGN KEY (id_produit) REFERENCES Produits(id_produit)
49 );

```

DOSSIER PROFESSIONNEL (DP)

```
-- Table Commandes
CREATE TABLE Commandes (
    id_command INT AUTO_INCREMENT PRIMARY KEY,
    id_user INT,
    date_commande TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    statut VARCHAR(20),
    total DECIMAL(10, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (id_user) REFERENCES User(id_user)
);

-- Table Detail_Command
CREATE TABLE Detail_Command (
    id_command INT,
    id_produit INT,
    quantité INT,
    prix DECIMAL(10, 2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id_command, id_produit),
    FOREIGN KEY (id_command) REFERENCES Commandes(id_command),
    FOREIGN KEY (id_produit) REFERENCES Produits(id_produit)
);
```

2. Précisez les moyens utilisés :

Pour réaliser le MCD, j'ai utilisé draw.io puis pour la réalisation du MPD.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ► *École Laplateforme*

Chantier, atelier, service ► *Projet personnel réalisé en cours de formation.*

Période d'exercice ► Du : *24/06/2024* au : *09/10/2025*

5. Informations complémentaires (facultatif)

Documents de conception: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Le projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>https

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 6 ▶ Développer les composants d'accès aux données.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Toujours dans le cadre du projet Game Zone, j'ai implémenté l'architecture avec laravel et inertia. J'ai utiliser d'ORM et d'effectuer des requêtes SQL avec Eloquent.

1. Initialisation de la connexion

L'initialisation de la connexion se fait simplement en ouvrant le fichier database.sqlite avec SQLite, que je peux explorer grâce à SQLite Viewer.

```
database > migrations > 2025_07_05_071146_create_categories_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  class CreateCategoriesTable extends Migration
8  {
9      public function up()
10     {
11         Schema::create('categories', function (Blueprint $table) {
12             $table->id();
13             $table->string('nom');
14             $table->string('slug')->default('default-slug')->change();
15             $table->foreignId('parent_id')->nullable()->constrained('categories')->onDelete('cascade');
16             $table->timestamps();
17         });
18     }
19
20     public function down()
21     {
22         Schema::dropIfExists('categories', function (Blueprint $table) {
23             $table->string('slug')->nullable()->change();
24         });
25     }
26 }
27 }
```

2. Utilitaire de construction et d'exécution de requêtes.

J'utilise SQLite Viewer comme utilitaire de construction et d'exécution de requêtes, ce qui me permet de tester et lire facilement les données de ma base database.sqlite

Cette fonction store permet de valider les données envoyées depuis un formulaire, d'enregistrer un nouveau produit avec son image, puis d'associer ses plateformes et leurs prix avant de rediriger vers la liste des produits avec un message de succès.

```

90
91 public function store(Request $request)
92 {
93     $validated = $request->validate([
94         'nom' => 'required|string|max:255',
95         'description' => 'required|string',
96         'image' => 'required|image|mimes:jpg,jpeg,png|max:2048',
97         'stock' => 'required|integer|min:0',
98         'category_id' => 'required|exists:categories,id',
99         'plateformes' => 'required|array',
100         'plateformes.*' => 'exists:plateformes,id',
101         'prix_plateformes' => 'required|array',
102         'prix_plateformes.*' => 'required|numeric|min:0',
103     ]);
104     if ($request->hasFile('image')) {
105         $imagePath = $request->file('image')->store('products', 'public');
106         $validated['image'] = $imagePath;
107     }
108
109     $produit = Produits::create([
110         'nom' => $validated['nom'],
111         'description' => $validated['description'],
112         'image' => $validated['image'],
113         'prix' => 0,
114         'stock' => $validated['stock'],
115         'category_id' => $validated['category_id'],
116     ]);
117
118     $pivotData = [];
119     foreach ($validated['plateformes'] as $plateformeId) {
120         $pivotData[$plateformeId] = [
121             'prix' => $validated['prix_plateformes'][$plateformeId]
122         ];
123     }
124     $produit->plateformes()->sync($pivotData);
125
126     return redirect()->route('products.index')->with('success', 'Produit créé avec succès !');
127 }

```

Ces méthodes permettent d'afficher soit le détail d'un produit précis avec sa catégorie et ses prix par plateforme, soit de filtrer la liste des produits selon la plateforme choisie (Nintendo, PlayStation ou Xbox) pour les afficher sur la page d'accueil connectée.

```

30 public function show($id)
31 {
32     $produit = Produits::with(['category', 'plateformes'])->findOrFail($id);
33
34     return Inertia::render('DetailProduit', [
35         'produit' => $produit,
36         'prixParPlateforme' => $produit->plateformes->pluck('pivot.prix', 'nom'),
37     ]);
38 }
39
40 public function showNintendo()
41 {
42     $produits = Produits::whereHas('plateformes', function ($query) {
43         $query->where('nom', 'Nintendo Switch');
44     })->get();
45
46     return Inertia::render('AccueilConnecter', compact('produits'));
47 }
48
49 public function showPlayStation()
50 {
51     $plateformes = ['PS4', 'PS5'];
52
53     $produits = Produits::whereHas('plateformes', function ($query) use ($plateformes) {
54         $query->whereIn('nom', $plateformes);
55     })->get();
56
57     return Inertia::render('AccueilConnecter', compact('produits'));
58 }
59
60 public function showXbox()
61 {
62     $plateformes = ['Xbox One', 'Xbox Serie X', 'Xbox Serie S'];
63
64     $produits = Produits::whereHas('plateformes', function ($query) use ($plateformes) {
65         $query->whereIn('nom', $plateformes);
66     })->get();
67
68     return Inertia::render('AccueilConnecter', compact('produits'));
69 }

```


DOSSIER PROFESSIONNEL (DP)

Cette méthode `search` permet de récupérer la chaîne de recherche transmise en paramètre de la requête, puis de filtrer les produits dont le nom contient cette chaîne via une requête "like", et enfin de retourner les résultats au format JSON. Cette fonction est généralement utilisée pour une recherche dynamique

```

70
71     public function search(Request $request)
72     {
73         $search = $request->query('search', '');
74         $produits = Produits::where('nom', 'like', "%{$search}%")->get();
75
76         return response()->json($produits);
77     }
78

```

Ces deux méthodes de contrôleur concernent la mise à jour et la suppression d'un produit. La méthode `update` valide les données reçues, puis met à jour les champs spécifiés du produit dans la base de données avant de rediriger vers la liste des produits avec un message de confirmation. La méthode `destroy` supprime simplement le produit sélectionné et redirige aussi vers la liste avec un message de succès. Ces actions correspondent aux opérations classiques de mise à jour (Update) et suppression (Delete) dans un CRUD Laravel

```

127
128     public function update(Request $request, Produits $produits)
129     {
130         $request->validate([
131             'nom' => 'required',
132             'prix' => 'required|numeric',
133             'stock' => 'required|integer|min:0',
134             'description' => 'nullable|string',
135             'image' => 'nullable|string',
136         ]);
137         $produits->update($request->only('nom', 'prix', 'stock', 'description', 'image'));
138         return redirect()->route('products.index')->with('success', 'Produit modifié !');
139     }
140
141     public function destroy(Produits $produits)
142     {
143         $produits->delete();
144         return redirect()->route('products.index')->with('success', 'Produit supprimé !');
145     }
146

```

```

79
80     public function create()
81     {
82         $categories = Categories::all();
83         $plateformes = Plateforme::all();
84
85         return Inertia::render('CreerProduit', [
86             'categories' => $categories,
87             'plateformes' => $plateformes,
88         ]);
89     }
90

```

J'ai développé un système de gestion de panier permettant à chaque utilisateur authentifié d'ajouter, modifier ou supprimer des produits. Grâce à Laravel et Eloquent, je récupère les produits liés à l'utilisateur et je calcule automatiquement le prix total de chaque ligne, que j'affiche ensuite côté client avec Inertia et React. »

```
1 <?php
2
3 namespace App\Http\Controllers;
4 use Illuminate\Support\Facades\DB;
5 use App\Models\Comandes;
6 use App\Models\Paniers;
7 use Illuminate\Http\Request;
8 use Inertia\Inertia;
9
10 class PanierController extends Controller
11 {
12     public function ajouterAuPanier(Request $request)
13     {
14         $request->validate([
15             'produit_id' => 'required|exists:produits,id',
16             'prix' => 'required|numeric|min:0',
17             'quantite' => 'required|integer|min:1',
18         ]);
19
20         $userId = auth()->id();
21         $panier = Paniers::where('user_id', $userId)
22             ->where('produit_id', $request->produit_id)
23             ->first();
24
25         if ($panier) {
26             $panier->quantite += $request->quantite;
27             $panier->prix = $request->prix;
28             $panier->save();
29         } else {
30             Paniers::create([
31                 'user_id' => $userId,
32                 'produit_id' => $request->produit_id,
33                 'quantite' => $request->quantite,
34                 'prix' => $request->prix,
35             ]);
36         }
37         return redirect()->route('panier.afficher');
38     }
39 }
```

```
40
41 public function afficher()
42 {
43     $panier = Paniers::where('user_id', auth()->id())
44         ->with('produit')
45         ->get();
46
47     $panierDetails = $panier->map(function ($item) {
48         $item->prix_total = $item->quantite * $item->prix;
49         return $item;
50     });
51
52     return Inertia::render('Panier', [
53         'panier' => $panierDetails,
54     ]);
55
56 public function supprimer($id)
57 {
58     $panier = Paniers::where('id', $id)
59         ->where('user_id', auth()->id())
60         ->first();
61
62     if ($panier) {
63         $panier->delete();
64         return redirect()->back()->with('success', 'Article supprimé du panier.');

```

```
1 <?php
2
3 namespace App\Http\Controllers;
4 use Illuminate\Http\Request;
5 use App\Models\Paniers;
6 use App\Models\Comandes;
7 use App\Models\Utilisateurs;
8 use Inertia\Inertia;
9
10 class CommandeController extends Controller
11 {
12     public function afficherPagePaiement()
13     {
14         $userId = auth()->id();
15         $panier = Paniers::where('user_id', $userId)
16             ->with('produit')
17             ->get();
18
19         if ($panier->isEmpty()) {
20             return redirect()->route('panier.afficher')->with('error', 'Votre panier est vide.');

```

Ce contrôleur gère la finalisation de la commande : il affiche la page de paiement avec le récapitulatif du panier et le total à régler, puis il valide la commande en enregistrant les informations du client ainsi que les produits achetés

2. Précisez les moyens utilisés :

Dans le cadre de ce projet, et pour l'accès aux données j'ai utilisé les modules node-postgres, node-postgres-format, et qs pour parser la query string.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association

École Laplateforme

Chantier, atelier, service

Projet personnel réalisé en cours de formation.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 7 ► Développer la partie back-end d'une application web ou web mobile.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'initialisation du serveur consiste à mettre en place l'environnement nécessaire pour exécuter l'application. J'ai configuré un serveur local avec PHP, Composer et Node.js, puis installé les dépendances Laravel et React via composer install et npm install. Le serveur de développement Laravel est lancé avec la commande php artisan serve, tandis que la partie front-end est compilée avec npm run dev. Cela permet d'avoir une application opérationnelle en local, prête à être déployée sur un serveur de production. »

Le chemin de la requête au niveau des routes est le suivant:

Client → Inertia.js (côté client) → Requête HTTP → Route Laravel → Contrôleur Laravel → Base de données / Traitement → Réponse Inertia → React actualise l'UI

1. Initialisation du serveur

```
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:yB4zJGbZmB9LtelkGwnFL4S0ihsAP9lKXo5jFr6VOe8=
4 APP_DEBUG=true
5 APP_URL=http://localhost
```

```
19
20 DB_CONNECTION=sqlite
```

```
SESSION_DRIVER=database
```

```
LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug
```

```
MAIL_MAILER=log
```

```
66 AWS_ACCESS_KEY_ID=
67 AWS_SECRET_ACCESS_KEY=
68 AWS_DEFAULT_REGION=us-east-1
69 AWS_BUCKET=
70 AWS_USE_PATH_STYLE_ENDPOINT=false
71
72 VITE_APP_NAME="${APP_NAME}"
73 ADMIN_REGISTRATION_CODE=Super_Nova23
74
```

```
PS C:\Users\Utilisateur\Desktop\E-commerceJeuxVideo> php artisan serve
```

```
INFO Server running on [http://127.0.0.1:8000].
```

```
Press Ctrl+C to stop the server
```

Installation de Composer et Node.js

Création du fichier .env (configuration de la base de données, clés d'API, etc.)

Migration des tables (php artisan migrate)

Lancement du serveur (php artisan serve)

2. Panier.jsx

```
7
8 export default function Panier() {
9   const { panier } = usePage().props;
10
11   function handleDelete(id) {
12     router.delete(`/panier/${id}`, {
13       preserveScroll: true,
14     });
15   }
16 }
```

Je déclare ensuite la route dans le fichier /routes/web.php le

```
120 Route::middleware('auth')->post('/ajouter-au-panier', [PanierController::class, 'ajouterAuPanier'])->name('panier.ajouter');
121
122 Route::get('/panier', [PanierController::class, 'afficher'])->name('panier.afficher');
123
124 Route::delete('/panier/{id}', [PanierController::class, 'supprimer'])->name('panier.supprimer');
125
126 Route::middleware(['auth'])->group(function () {
127   Route::get('/paiement', [CommandeController::class, 'afficherPagePaiement'])->name('paiement.page');
128   Route::post('/paiement/valider', [CommandeController::class, 'validerCommande'])->name('paiement.valider');
129 });
130
```



DOSSIER PROFESSIONNEL (DP)

Période d'exercice ► Du : 24/06/2024 au : 09/10/2025

5. Informations complémentaires (facultatif)

Le repo du projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Le projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

3. Router d'entité

```
Route::get('/forgot-password', [PasswordResetLinkController::class, 'create'])
    ->middleware('guest')
    ->name('password.request');

Route::middleware(['throttle:5,1'])->group(function () {
    Route::post('/forgot-password', [PasswordResetLinkController::class, 'store'])
        ->middleware('guest')
        ->name('password.email');
});
```

Contrairement à l'exemple NodeJS avec `links.route.js` qui définit tout un CRUD, ton contrôleur gère un cas métier précis : la demande de réinitialisation de mot de passe.

Il n'a donc pas besoin de méthodes `index`, `update` ou `destroy`.

L'organisation Laravel sépare les responsabilités en plusieurs contrôleurs (`PasswordResetLinkController` pour l'envoi du lien, `NewPasswordController` pour enregistrer le nouveau mot de passe, etc.).

Dans ce fichier `/routes/web.php` je vais déclarer les endpoints, avec les méthodes associées (`get`, `post`). On remarque l'utilisation de middlewares, que j'expliquerais plus tard.

4. Controller

Une fois la requête effectuée, c'est le controller qui va prendre le relais et traiter la partie "logique" de notre API, il va traiter les données de la requête puis interagir avec la couche des données de notre API et enfin renvoyer des données en retour. En voici un exemple avec le controller de création de lien, contenu dans le fichier `app/Http/Controllers/Auth/PasswordResetLinkController.php`

```
http://Controllers/Auth/PasswordResetLinkController.php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Password;
use Inertia\Inertia;
use Inertia\Response;

class PasswordResetLinkController extends Controller
{
    /**
     * Show the password reset link request page.
     */
    public function create(Request $request): Response
    {
        return Inertia::render('auth/forgot-password', [
            'status' => $request->session()->get('status'),
        ]);
    }

    /**
     * Handle an incoming password reset link request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'email' => 'required|email',
        ]);

        Password::sendResetLink(
            $request->only('email')
        );

        return back()->with('status', __('A reset link will be sent if the account exists.));
    }
}
```

Une fois la requête effectuée via les routes définies dans `routes/web.php`, c'est le controller qui prend le relais. Dans mon projet, le contrôleur `PasswordResetLinkController` se charge de la logique liée à la demande de réinitialisation de mot de passe.

Il possède deux méthodes principales :

create : qui retourne la page Inertia `auth/forgot-password` contenant le formulaire de saisie de l'email.

store : qui valide l'email saisi par l'utilisateur, appelle la façade `Password` de Laravel pour envoyer un lien de réinitialisation, puis renvoie un message de confirmation.

Ce contrôleur illustre bien le rôle d'un controller en Laravel : il reçoit la requête HTTP, applique la logique métier (validation, envoi de mail), et retourne soit une vue Inertia, soit une redirection avec un statut.

5. Middlewares

Pour assurer la gestion de l'authentification d'un utilisateur ou pour valider les données, j'ai créé des middleware que je passe au niveau du router, en voici un exemple:

```
public function register(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|confirmed|min:6',
        'role' => 'nullable|in:user,admin',
        'admin_code' => 'nullable|string|max:255|required_if:role,admin',
    ]);

    $role = 'user';
    if ($request->role === 'admin' && $request->admin_code === env('ADMIN_REGISTRATION_CODE')) {
        $role = 'admin';
    }

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'role' => $role,
    ]);

    Auth::login($user);

    return $role === 'admin'
        ? redirect()->route('dashboard')
        : redirect()->route('accueil.connecter');
}
```

Pour gérer l'authentification de l'utilisateur, j'ai créé le middleware suivant:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\Auth;

class AdminMiddleware
{
    public function handle($request, Closure $next)
    {
        // // Adapte la condition selon ton modèle User
        // if (!Auth::check() || !Auth::user()->{
        //     abort(403, 'Accès refusé');
        // }

        return $next($request);
    }

    public function boot()
    {
        Inertia::share('auth', function () {
            return [
                'user' => Auth::user(),
            ];
        });
    }
}
```

Dans mon projet, j'utilise également des contrôleurs et middlewares pour gérer la vérification des utilisateurs. Par exemple, le VerifyEmailController est déclenché lorsqu'un utilisateur clique sur le lien de confirmation de son adresse e-mail.

Ce contrôleur joue le rôle d'un middleware de sécurité : il vérifie si l'utilisateur a déjà validé son adresse, et si ce n'est pas le cas, il la marque comme vérifiée et enregistre l'action via l'événement Verified.

Enfin, l'utilisateur est redirigé vers son tableau de bord avec un paramètre de confirmation.

Ce mécanisme montre bien que, comme dans l'exemple NodeJS où un middleware valide une requête avant de continuer, Laravel dispose de contrôleurs et middlewares qui assurent la validation et la sécurité avant d'exécuter la logique métier.

```
app > Http > Controllers > Auth > VerifyEmailController.php
1  <?php
2
3  namespace App\Http\Controllers\Auth;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Auth\Events\Verified;
7  use Illuminate\Foundation\Auth\EmailVerificationRequest;
8  use Illuminate\Http\RedirectResponse;
9
10 class VerifyEmailController extends Controller
11 {
12     /**
13      * Mark the authenticated user's email address as verified.
14      */
15     public function __invoke(EmailVerificationRequest $request): RedirectResponse
16     {
17         if ($request->user()->hasVerifiedEmail()) {
18             return redirect()->intended(route('dashboard', absolute: false).'?verified=1');
19         }
20
21         if ($request->user()->markEmailAsVerified()) {
22             /** @var \Illuminate\Contracts\Auth\MustVerifyEmail $user */
23             $user = $request->user();
24
25             event(new Verified($user));
26         }
27
28         return redirect()->intended(route('dashboard', absolute: false).'?verified=1');
29     }
30 }
31
```


En Laravel : la validation se fait directement dans le controller (ou via une classe dédiée FormRequest).

```

29 public function store(Request $request): RedirectResponse
30 {
31     $request->validate([
32         'email' => 'required|email',
33     ]);
34
35     Password::sendResetLink(
36         $request->only('email')
37     );
38
39     return back()->with('status', __('A reset link will be sent if the account exists.'));
40 }
41
42

```

```

<?php

namespace App\Exceptions;

use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
use Throwable;

class Handler extends ExceptionHandler
{
    /**
     * A list of the exception types that are not reported.
     *
     * @var array<int, class-string<Throwable>>
     */
    protected $dontReport = [];

    /**
     * A list of the inputs that are never flashed to the session on validation exceptions.
     *
     * @var array<int, string>
     */
    protected $dontFlash = [
        'current_password',
        'password',
        'password_confirmation',
    ];

    /**
     * Register the exception handling callbacks for the application.
     */
    public function register(): void
    {
        $this->reportable(function (Throwable $e) {
            //
        });
    }
}

```

Afin de gérer de manière centralisée les erreurs dans mon projet, j'utilise le fichier `app/Exceptions/Handler.php`. Ce gestionnaire global permet d'intercepter toutes les erreurs qui surviennent dans l'application (par exemple une validation échouée, une ressource introuvable, ou une exception système). On peut ainsi personnaliser la réponse envoyée : renvoyer une erreur JSON dans le cas d'un appel API, ou afficher une page d'erreur dans le cas d'une requête web classique.

6. Rate limiting

```
Route::get('/forgot-password', [PasswordResetLinkController::class, 'create'])
    ->middleware('guest')
    ->name('password.request');

Route::middleware(['throttle:5,1'])->group(function () {
    Route::post('/forgot-password', [PasswordResetLinkController::class, 'store'])
        ->middleware('guest')
        ->name('password.email');
});
```

Cette route POST /forgot-password utilise le middleware throttle pour limiter les requêtes à 5 par minute afin de protéger la fonctionnalité de réinitialisation de mot de passe. Le middleware guest s'assure que seuls les utilisateurs non connectés peuvent y accéder, et la route est nommée password.email. »

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Facades\Route;
use Illuminate\Foundation\Support\Providers\RouteServiceProvider as ServiceProvider;
use Illuminate\Http\Request;

class RouteServiceProvider extends ServiceProvider
{
    /**
     * Path to the "home" route for your application.
     */
    public const HOME = '/home';

    /**
     * Register any application services.
     */
    public function register(): void
    {
        //
    }

    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        $this->configureRateLimiting();
        $this->routes(function () {
            Route::middleware('web')
                ->group(base_path('routes/web.php'));

            Route::middleware('api')
                ->prefix('api')
                ->group(base_path('routes/api.php'));
        });

        /**
         * Configure the rate limiters for the application.
         */
        protected function configureRateLimiting(): void
        {
            // Limiter par défaut pour les API
            RateLimiter::for('api', function (Request $request) {
                return Limit::perMinute(60)->by($request->user()?->id ?: $request->ip());
            });

            // Limiter pour le mot de passe oublié
            RateLimiter::for('forgot-password', function (Request $request) {
                return Limit::perMinute(5)->by($request->ip());
            });
        }
    }
}
```

Ce RouteServiceProvider configure les routes web et API de l'application et définit des limitations de débit (rate limiting) : un limiteur global pour les API à 60 requêtes par minute et un limiteur spécifique pour la fonctionnalité de réinitialisation de mot de passe, limité à 5 requêtes par minute par adresse IP.



DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Pour construire ce projet j'ai utilisé les librairies suivantes:

- [RouteServiceProvider](#): pour gérer le rate limiting.
- [Controller](#) pour construire et valider les tokens d'authentification
- [Controller et Middleware](#) pour gérer la validation des données

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet

4. Contexte

Nom de l'entreprise, organisme ou association ► *École Laplateforme*

Chantier, atelier, service ► *Projet personnel réalisé en cours de formation.*

Période d'exercice ► Du : *24/06/2024* au : *09/10/2025*

5. Informations complémentaires (facultatif)

Le repo de projet est accessible à cette adresse: <https://github.com/aliciachabane/E-commerceJeuxVideo>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

CP 8 ► *Élaborer et mettre en œuvre des composants dans une application de gestion de contenu ou e-commerce.*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Création des migrations (tables), Relation entre les tables (models), création des seeders, création des Controllers et des méthodes (fonctionnalités)

2. Précisez les moyens utilisés :

Sqlite, Laravel

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet

4. Contexte

Nom de l'entreprise, organisme ou association ►

Chantier, atelier, service ►

Période d'exercice ► Du : 24/06/2024 Au : 09/10/2025

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date

Déclaration sur l'honneur

Je soussigné(e) Alicia Chabane ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à Cannes le 14/06/2025

pour faire valoir ce que de droit.

Signature : 



DOSSIER PROFESSIONNEL (DP)

MINISTÈRE CHARGÉ
DE L'EMPLOI

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé

ANNEXES

(Si le RC le prévoit)